Experiment - 4

Aim:

Using the employee schema, write queries with INNER JOIN, self-join, 3-way join, correlated subqueries, EXISTS and simulated INTERSECT and EXCEPT; and compare execution plans using EXPLAIN.

Source Code

1. INNER JOIN

```
SELECT e.emp_name, d.dept_name, d.location
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id
ORDER BY d.dept_name;
```

2. LEFT JOIN

```
SELECT e.emp_name, w.proj_id, w.hours
FROM Employee e
LEFT JOIN Works_On w ON e.emp_id = w.emp_id
ORDER BY e.emp_id;
```

3. Self-Join (Employee ↔ Manager)

```
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM Employee e
LEFT JOIN Employee m ON e.manager_id = m.emp_id
ORDER BY e.emp_id;
```

4. Three-way Join

```
SELECT e.emp_name, d.dept_name, p.proj_name, w.hours
FROM Employee e
JOIN Works_On w ON e.emp_id = w.emp_id
JOIN Project p ON w.proj_id = p.proj_id
JOIN Department d ON e.dept_id = d.dept_id
WHERE d.dept_name = 'IT'
ORDER BY e.emp_name;
```

5. Correlated Subquery

```
SELECT e.emp_name, e.salary, e.dept_id
FROM Employee e
WHERE e.salary > (SELECT AVG(salary)
                  FROM Employee x
                  WHERE x.dept_id = e.dept_id)
ORDER BY e.dept_id;
```

6. Exists

```
SELECT e.emp_id, e.emp_name
FROM Employee e
WHERE EXISTS (SELECT 1 FROM Works_On WHERE w.emp_id =
e.emp_id);

SELECT d.dept_name
FROM Department d
WHERE NOT EXISTS (SELECT 1 FROM Employee e WHERE
e.dept_id = d.dept_id);
```

7. Simulated INTERSECT

```
-- Employee who are managers AND also work on at least
   one project
SELECT emp_id, emp_name FROM Employee
WHERE emp_id IN (SELECT manager_id FROM Employee
WHERE manager_id is NOT NULL)
INTERSECT -- not supported in MySQL
SELECT emp_id, emp_name FROM Employee;
```

8. Simulated EXCEPT

```
SELECT e.emp_id, e.emp_name
FROM Employee e
WHERE e.emp_id NOT IN (SELECT w.emp_id FROM Works_On w);
```

-- Employees of IT who are NOT on project 107

```
SELECT e.emp_id, e.emp_name
FROM Employee e
WHERE e.dept_id = 3
  AND e.emp_id NOT IN (SELECT w.emp_id FROM Works_On
w WHERE w.proj_id = 107);
```

9. EXPLAIN - Execution Plan Comparison

Plan A - Three-way join

```
EXPLAIN
SELECT e.emp_name, p.proj_name
FROM Employee e
JOIN Works_On w ON e.emp_id = w.emp_id
JOIN Project p ON w.proj_id = p.proj_id
WHERE p.proj_name = 'Website Revamp';
```

Plan B - Equivalent EXISTS/correlated subquery

```
EXPLAIN
SELECT e.emp_name
FROM Employee e
WHERE EXISTS (SELECT 1
              FROM Works_On w
              JOIN Project p ON w.proj_id = p.proj_id
              WHERE w.emp_id = e.emp_id
                AND p.proj_name = 'Website Revamp');
```

Plan C - INNER JOIN vs LEFT JOIN

```
EXPLAIN SELECT e.emp_name, d.dept_name
FROM Employee e INNER JOIN Department d ON e.dept_id
= d.dept_id;
EXPLAIN SELECT e.emp_name, d.dept_name
FROM Employee e LEFT JOIN Department d ON e.dept_id
= d.dept_id;
```

Indexes to add before re-running the plans:

```
CREATE INDEX idx_emp_dept ON Employee(dept_id);
CREATE INDEX idx_wo_proj ON works_On(proj_id);
CREATE INDEX idx_proj_name ON Project(proj_name);
```

Output

Query → Result Summary

INNER JOIN → 30 rows - every employee matched with their department

LEFT JOIN (Emp-works_On) → 32 rows; Farhan(6) and Sneha(18) show NULL proj_id/hours

LEFT JOIN (Dept-Emp) → 5 rows, emp_count = 6 per department

Self-Join → 30 rows; 5 managers show NULL manager

3-way Join → 5 rows - IT employees on projects with hours

Correlated Subquery → Employees earning above their own department average

EXISTS → 28 rows (excludes emp 6 and 18); NOT EXISTS on Department → empty

Simulated INTERSECT → 5 managers (1, 7, 13, 19, 25) who also work on projects

Simulated EXCEPT → 2 rows - Farhan(6), Sneha(18)

EXPLAIN comparison (typical 8 output of MySQL):

Plan	table	type	Key	rows	Extra
A	e	ALL	NULL	30	Using WHERE
A	w	ref	PRIMARY	1	Using INDEX
A	p	eq-ref	PRIMARY	1	Using WHERE
B	e	ALL	NULL	30	Using WHERE
B	w	ref	PRIMARY	1	Using WHERE; Using INDEX
B	p	eq-ref	PRIMARY	1	Using WHERE
C (INNER)	e	ALL	NULL	30	-
C (INNER)	d	eq-ref	PRIMARY	1	-
C (LEFT)	e	ALL	NULL	30	-
C (LEFT)	d	eq-ref	PRIMARY	1	-

Observations:

- The 3-way JOIN is expected as a nested-loop join; Project is accessed with eq-ref (primary key lookup) → 1 row per outer row.
- The EXISTS version produces a DEPENDENT SUBQUERY; it stops at the first matching row, so it is efficient when the outer result is small, but can be slower where the inner table is large.
- LEFT JOIN and INNER JOIN produced identical plans here because every employee has a valid dept_id; the optimiser cannot drop the outer-join semantics, but the access path is unchanged.
- After creating idx_emp_dept and idx_wo_proj, the type column changes from ALL to ref/index, and the rows estimate drops sharply - confirming index usage.